

OpenEd AI Tutor Product and Technical Spec

Executive summary

BYOK gives OpenEd access to an LLM brain, but it does not automatically create a tutor. A tutor is a pedagogy system: it knows the course, diagnoses the learner's intent, chooses whether to explain, hint, ask a question, draw, quiz, or escalate, and it stays aligned to learning outcomes.

The AI Tutor is one of OpenEd's most important proprietary systems. The v1 implementation should be useful even if some advanced pieces are stubbed, but the architecture should avoid exposing the long-term tutor engine in frontend source code.

Critical architecture truth

Client-side BYOK and proprietary secret tutoring logic are in tension.

If the tutor runs fully in the browser, any prompts, policies, and orchestration sent to the LLM can be inspected by users. If OpenEd wants to keep the tutoring engine proprietary, the tutoring orchestration must run on a backend/gateway. In that model, the learner's key can be sent ephemerally to the gateway for a request, not stored or logged, but it is no longer strictly "client-side only."

OpenEd has three options:

Option	How it works	Pros	Cons
Fully client-side BYOK	Browser stores key and calls provider directly	Lowest platform cost; transparent; open-source friendly	Prompts exposed; weaker safety; harder analytics/control
Server tutor gateway with BYOK passthrough	Browser sends key per request; server uses hidden tutor policy and does not store key	Proprietary tutor logic; safer orchestration; better QA	User key transits server; requires strong trust and no-log design
Hybrid	Basic open client tutor; advanced tutor gateway for managed mode	Flexible; supports open/free and proprietary advanced mode	More complexity

Recommended v1:

- Start with hybrid.

- Client-side BYOK for simple tutor mode.
- Keep the tutor protocol open enough to trust.
- Keep advanced pedagogy orchestration server-side for future managed mode.
- Be explicit in product copy about which mode is active.

Tutor mission

The tutor should help learners think, not just answer.

It should:

- Explain concepts using course context.
- Give examples and analogies.
- Ask Socratic follow-up questions.
- Provide hints before answers.
- Connect answers to lesson objectives.
- Help fill artifact fields without doing the whole artifact.
- Explain feedback and rubrics.
- Generate simple diagrams where possible.
- Admit uncertainty and cite sources.

It should not:

- Pretend to know unsupported facts.
- Invent course sources.
- Give final answers during assessments without policy permission.
- Store learner API keys on OpenEd servers.
- Replace educator review for high-stakes grading.
- Make medical/legal/financial claims outside course scope.

Tutor modes

Mode	Purpose	Behavior
Explain	Clarify concepts	Direct but concise explanation with source reference.
Socratic	Build reasoning	Ask guiding questions; no immediate answer.
Hint	Support practice	Give one small next step.
Diagram	Visual explanation	Return graph/steps/overlay instructions.
Quiz me	Retrieval practice	Ask question, wait, then feedback.

Mode	Purpose	Behavior
Artifact coach	Help with proof	Explain field meaning and suggest improvements.
Source guide	Further learning	Recommend source card and why.
Safety/uncertainty	Unknown or risky	Refuse unsupported answer and suggest source/educator.

Tutor pipeline

```

User message
-> context collector
-> intent classifier
-> safety/policy check
-> retrieval from lesson/course/source/rubric state
-> pedagogical strategy selector
-> response generator
-> citation/uncertainty checker
-> UI renderer
-> feedback logger

```

Context collector

Context inputs:

- Course ID.
- Module ID.
- Lesson ID.
- Current tab.
- Current video time, if available.
- Transcript segment.
- Lesson summary.
- Key ideas.
- Source cards.
- Glossary terms.
- Practice prompt.
- Artifact rubric.
- Learner progress.
- Previous tutor turns in session.

Do not include:

- API keys in logs.

- Hidden admin notes.
- Other learners' private artifacts.
- Sensitive personal data.

Intent classifier

Intents:

- Concept explanation.
- Simpler explanation.
- Example request.
- Diagram request.
- Video moment question.
- Quiz/help request.
- Artifact field help.
- Rubric feedback help.
- Source recommendation.
- Off-topic question.
- Unsafe/academic-integrity risk.

Pedagogical strategies

The tutor should choose from strategies:

- Explain-then-check.
- Ask-before-answer.
- Worked example.
- Contrast weak/strong example.
- Error diagnosis.
- Progressive hint.
- Retrieval practice.
- Reflection prompt.
- Source handoff.

Visual/video question capability

The "ask about the visual in the video" experience is impressive but technically complex.

For OpenEd, support it in phases:

V1: Visual anchor cards

Educator attaches visual descriptions to lesson chapters:

- Timestamp.
- Visual description.
- Concepts shown.
- Diagram labels.
- Expected questions.

Tutor can answer questions about these visual anchors, not the raw video frame.

V1.1: OpenEd-hosted frame snapshots

For videos uploaded/owned by OpenEd or the educator with permission:

- Capture key frames.
- Run OCR/vision captioning server-side.
- Store scene metadata.
- Link to transcript and concept map.
- Tutor can refer to the scene.

V2: Drawing overlay tutor

For visual explanations:

- Tutor returns structured drawing commands.
- Frontend renders arrows, highlights, labels, shapes.
- Learner can ask "why this step?"
- Educator can pre-approve diagrams for key concepts.

Example response contract:

```
{
  "mode": "visual_explain",
  "target": "video_frame_or_diagram_id",
  "explanation": "...",
  "drawings": [
    {"type": "arrow", "from": "pointA", "to": "pointB", "label": "hypotenuse"},
    {"type": "highlight", "target": "triangle_right_angle"}
  ],
  "check_question": "Which side is opposite the right angle?"
}
```

Important: for external YouTube/reference videos, OpenEd cannot assume legal or technical ability to capture frames. Use author-provided summaries and visual anchors unless content is hosted/owned/permissioned.

Prompt secrecy strategy

What can be open:

- High-level tutor principles.
- Safety policy.
- Data model.
- UI components.
- Client adapter interface.

What should be private if OpenEd wants proprietary advantage:

- Detailed tutor prompt library.
- Intent classifier examples.
- Rubric-to-feedback prompt chains.
- Tutor evaluation datasets.
- Failure-mode datasets.
- Strategy selection heuristics.
- Scoring calibration prompts.

If the project chooses open source, publish the tutor protocol and invite community contributions. If the project chooses proprietary mode, run orchestration server-side and avoid shipping sensitive prompts in the frontend.

BYOK modes

Session-only key

- Key held in memory.
- Lost on refresh/close.
- Safest BYOK option.

Local browser storage

- Optional.
- Requires explicit warning.
- Uses localStorage or IndexedDB.

- Learner can delete key any time.
- Not a secure vault.

Server passthrough

- User key sent with request.
- Never stored.
- Never logged.
- Redacted from traces.
- Useful if proprietary tutor logic is server-side.

Safety and integrity rules

- During quizzes, tutor gives hints first.
- During artifact submission, tutor can help improve but should not write final submission without learner contribution.
- Tutor should cite course sources or say when context is insufficient.
- Tutor should ask learners to attempt before revealing full solutions for assessment tasks.
- Tutor should provide uncertainty states.

Tutor evaluation

Track:

- Answer helpfulness rating.
- Source-grounding rate.
- Hallucination reports.
- Hint vs answer ratio.
- Tutor usage before lesson completion.
- Tutor usage before artifact submission.
- Learner confidence before/after.
- Educator review of tutor answers.
- Safety incident count.

Failure modes

Failure	Mitigation
Tutor becomes answer machine	Intent policy and assessment mode constraints.
Tutor hallucinates sources	Retrieval-only source answer mode and citation checks.
BYOK keys leak	Do not log; prefer session-only; warnings; backend redaction.
Prompt IP exposed	Do not ship proprietary prompts client-side.
Learner overrelies	Ask-to-think, hint mode, reflection prompts.
Visual answers are wrong	Use educator-approved visual anchors in v1.
Provider output varies	Model adapters and answer evaluation.

V1 acceptance criteria

- Tutor can answer from lesson context.
 - Tutor can refuse or caveat when context is insufficient.
 - Tutor has explain, hint, quiz, artifact help, and source guide modes.
 - BYOK setup has session and optional local modes.
 - No API key is logged or stored server-side.
 - Tutor answer ratings are captured.
 - Educators can configure suggested prompts for a lesson.
-

Source notes

This document is grounded in the current OpenEd/FME project context and the following reference set where applicable:

- Uploaded Frontier Model Evaluations prototype code bundle, latest inspected bundle: Design System for Frontier Model Evaluation (5).zip.
- Uploaded Frontier Model Evaluations 51-hour curriculum and Module 1 foundations files.
- Coursera, Udemy, DataCamp, Moodle, Khanmigo, Duolingo public documentation and product pages.
- Supabase Auth and Row Level Security documentation.
- OpenAI API key safety guidance, MDN Web APIs, and YouTube IFrame Player API documentation.

- Bloom's 2 sigma problem, intelligent tutoring systems research, CourseAssist, LearnLM-Tutor, DCCI, and UNESCO guidance for generative AI in education.

Document version: OpenEd v1 planning pack, 2026-06-17.